

Json come utilizzarlo in JS

Programmazione

Cos'è JSON in JavaScript

JSON (JavaScript Object Notation) è un formato testuale leggero utilizzato per rappresentare, organizzare e scambiare dati strutturati tra applicazioni e sistemi informatici.

La sua sintassi deriva dalla notazione degli oggetti JavaScript, ma JSON è un formato indipendente dal linguaggio di programmazione e può essere utilizzato anche con Python, Java, C# e molti altri linguaggi.



I dati vengono organizzati principalmente attraverso coppie chiave-valore, racchiuse tra parentesi graffe {}, oppure attraverso array delimitati da parentesi quadre [].

JSON supporta diversi tipi di valore, tra cui stringhe, numeri, booleani, valori null, oggetti e array.

A differenza degli oggetti JavaScript, le chiavi JSON devono essere racchiuse tra doppi apici e non è possibile inserire direttamente funzioni, valori undefined o commenti.

Questa struttura rende JSON facilmente leggibile sia dagli sviluppatori sia dai programmi che devono interpretarne il contenuto.



A cosa serve JSON in JavaScript

JSON viene utilizzato principalmente per la comunicazione e lo scambio di informazioni tra applicazioni, soprattutto nello sviluppo web e nell'interazione tra frontend e backend.

In JavaScript è particolarmente importante perché permette di ricevere dati da server e API REST, rappresentare informazioni relative a utenti, prodotti e ordini e memorizzare dati strutturati nel browser tramite strumenti come localStorage.



Principali metodi per utilizzare JSON in JavaScript

1. JSON.stringify() – Converte un oggetto o un array JavaScript in una stringa JSON. È utilizzato principalmente per salvare o trasmettere dati. Esempio: convertire le informazioni di uno studente in una stringa da inviare a un server.

2. JSON.parse() – Converte una stringa JSON valida in un oggetto o array JavaScript, permettendo di accedere alle proprietà e modificarne i valori. Esempio: ricevere dal server un elenco di prodotti in JSON e trasformarlo in un array utilizzabile.



Principali metodi per utilizzare JSON in JavaScript

3. JSON.stringify() con replacer – Permette di selezionare o trasformare le proprietà da includere durante la conversione in JSON. Esempio: salvare le informazioni di un utente escludendo determinati campi riservati. Non sostituisce i controlli di sicurezza sui dati.

4. JSON.stringify() con space – Permette di formattare la stringa JSON aggiungendo indentazioni e spaziature, rendendo il contenuto più leggibile. Esempio: visualizzare nella console un catalogo prodotti organizzato su più righe.



Metodi JavaScript utilizzati insieme a JSON

1. localStorage.setItem() – Salva una stringa nel browser associandola a una chiave. Viene spesso utilizzato insieme a JSON.stringify() per memorizzare oggetti. Esempio: salvare le preferenze di un utente anche dopo la chiusura della pagina.

2. localStorage.getItem() – Recupera una stringa precedentemente salvata nel browser, che può essere riconvertita con JSON.parse(). Esempio: recuperare i dati di un utente dopo il caricamento della pagina.



Metodi JavaScript utilizzati insieme a JSON

3. localStorage.removeItem() – Elimina un elemento salvato nel browser attraverso la sua chiave. Esempio: cancellare un carrello precedentemente memorizzato in formato JSON.

4. Response.json() – Legge il corpo di una risposta HTTP e ne interpreta il contenuto JSON, restituendo una Promise. Esempio: recuperare da un'API REST un elenco di utenti da visualizzare in una pagina web.



Nota didattica: gli unici due metodi principali dell'oggetto globale JSON sono JSON.stringify() e JSON.parse().

Gli altri elementi elencati sono parametri opzionali di questi metodi oppure funzionalità JavaScript utilizzate per memorizzare, ricevere e manipolare dati JSON.



JavaScript mette a disposizione due metodi fondamentali: JSON.stringify(), che converte un valore JavaScript compatibile in una stringa JSON, e JSON.parse(), che interpreta una stringa JSON valida trasformandola in un valore JavaScript utilizzabile nel programma.

Attraverso questi strumenti è possibile salvare, trasmettere e ricostruire informazioni mantenendone la struttura originale.

JSON rappresenta quindi uno standard fondamentale nello sviluppo di applicazioni web moderne, perché consente di separare la rappresentazione dei dati dalla logica applicativa e facilita l'integrazione tra sistemi sviluppati con tecnologie differenti.



Esercizio 1 – Gestione di uno studente con JSON

Realizzare un programma JavaScript che permetta di gestire le informazioni di uno studente attraverso un oggetto contenente nome, cognome, età, classe e voto medio.

Il programma deve creare l'oggetto con valori iniziali scelti liberamente, convertirlo in una stringa JSON attraverso il metodo JSON.stringify() e visualizzare il risultato nella console.

Successivamente, utilizzare JSON.parse() per riconvertire la stringa in un oggetto JavaScript, modificare il voto medio dello studente e visualizzare nuovamente tutte le informazioni aggiornate.



Spiegazione

Il programma crea inizialmente un oggetto JavaScript contenente le informazioni dello studente.

Il metodo JSON.stringify() trasforma l'oggetto in una stringa JSON, adatta alla memorizzazione o alla trasmissione.

Successivamente, JSON.parse() ricostruisce un oggetto JavaScript partendo dalla stringa.

A questo punto è possibile modificare una proprietà, come votoMedio, utilizzando la notazione con il punto.

Infine, console.log() permette di visualizzare singolarmente le informazioni aggiornate.

```
1.
2.let studente = {
3.    nome: "Luca",
4.    cognome: "Rossi",
5.    eta: 17,
6.    classe: "4A",
7.    votoMedio: 7.5
8.};
9.
10.// Conversione in stringa JSON
11.let testoJSON = JSON.stringify(studente);
12.console.log("Stringa JSON:", testoJSON);
13.
14.// Conversione da JSON a oggetto JavaScript
15.let studenteRecuperato = JSON.parse(testoJSON);
16.
17.// Modifica del voto
18.studenteRecuperato.votoMedio = 8;
19.
20.// Visualizzazione dei dati aggiornati
21.console.log("Studente aggiornato:");
22.console.log("Nome:", studenteRecuperato.nome);
23.console.log("Cognome:", studenteRecuperato.cognome);
24.console.log("Età:", studenteRecuperato.eta);
25.console.log("Classe:", studenteRecuperato.classe);
26.console.log("Voto medio:", studenteRecuperato.votoMedio);
27.
```


Buon davante a tutti